PolliNexus API — Manual Test Guide (mirrors pollinexus-done.ipynb)

Intro Speech (Data Science + Programming Understanding)

> I approach PolliNexus with a full-stack data mindset: define clear hypotheses, assess data quality, and apply domain-aware cleaning before modeling. I favor interpretable methods and report metrics honestly, highlighting feature importance and limitations. On the engineering side, I design secure, observable, and maintainable systems—validation on inputs, structured logging, health/metrics endpoints, and clear API contracts—so analysis scales from a notebook into reliable services.

Follow these steps to validate the API end-to-end. Replace placeholders like , , , <DATASET_ID>, <JOB_ID>, <VIS_ID>, <TASK_ID>.

Reference by Tag Group

- Health & Information: docs/api-groups/api-health-info.md
- User Authentication & Accounts: docs/api-groups/api-auth.md
- Dataset Management: docs/api-groups/api-datasets.md
- Data Analysis: docs/api-groups/api-analysis.md
- Visualizations: docs/api-groups/api-visualizations.md
- Monitoring: docs/api-groups/api-monitoring.md

0) Base setup (Makefile-driven)

> First, I'll spin up the API using the project's Makefile so the environment is reproducible and aligned with dev workflows.

- Local API (uv):

```
make run-local
```

- Docker API:

```
make run-docker
```

- Remote VM API:

```
make run-remote
```

1) Health, info, quick checks

> Before any workflow, I verify health, detailed status, and basic API info to ensure a clean baseline.

- Makefile health targets (local):

```
make health
make health-detailed
```

- Direct API checks:

```
curl "http://localhost:8000/api/v1/health"
curl "http://localhost:8000/api/v1/health/detailed"
curl "http://localhost:8000/api/v1/info"
curl "http://localhost:8000/api/v1/metrics"
```

2) (Optional) Auth flow to get a JWT

> If I want to demonstrate authentication, I'll register, verify OTP, and fetch a JWT, then call a protected endpoint.

- Register (sends OTP to email)

```
curl -X POST "http://localhost:8000/api/v1/user/register" \
  -H "Content-Type: application/json" \
  -d '{"full_name":"Test User","email":"<EMAIL>","phone":"+10000000000"}'
```

- Send verification code again if needed

```
curl -X POST "http://localhost:8000/api/v1/user/send-verification" \
  -H "Content-Type: application/json" \
  -d '{"email":"<EMAIL>","verification_type":"email"}'
```

- Verify registration (use the OTP you received); receives JWT

```
curl -X POST "http://localhost:8000/api/v1/user/verify-registration" \
  -H "Content-Type: application/json" \
  -d '{"email":"<EMAIL>","otp":"<OTP>"}'
```

- Login OTP (later, when logging in)

```
```

```
curl -X POST "http://localhost:8000/api/v1/user/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"<EMAIL>"}'
```

- Verify login to get JWT

```
curl -X POST "http://localhost:8000/api/v1/user/verify-login" \
  -H "Content-Type: application/json" \
  -d '{"email":"<EMAIL>","otp":"<OTP>"}'
```

- Check profile

```
curl -H "Authorization: Bearer <JWT>" "http://localhost:8000/api/v1/user/me"
```

3) Upload dataset (plants_and_bees.csv)

> Next, I'll upload the plants-and-bees dataset to enable downstream analysis and visualization.

```
curl -X POST "http://localhost:8000/api/v1/datasets/" \
  -F "name=Plants and Bees Dataset" \
  -F "description=Sample pollinator data" \
  -F "file=@dataset/plants_and_bees.csv"
```

4) Inspect datasets

> I confirm ingestion with list/get, then pull dataset info and a health check to validate integrity and stats.

```
# List
curl "http://localhost:8000/api/v1/datasets/?skip=0&limit=20"
# Get one by id
curl "http://localhost:8000/api/v1/datasets/<DATASET_ID>"
# Dataset info
curl "http://localhost:8000/api/v1/datasets/<DATASET_ID>/info"
# Dataset health
curl "http://localhost:8000/api/v1/datasets/<DATASET_ID>/health"
```

5) Start ML analysis

> With data in place, I trigger ML workflows (bee preferences, recommendations) to generate analytical insights.

- Bee preference model

```
curl -X POST "http://localhost:8000/api/v1/analysis/bee-preferences/" \
  -H "Content-Type: application/json" \
  -d '{"dataset_id": <DATASET_ID>, "target_column": "nonnative_bee", "model_type":
"random_forest", "test_size": 0.2}'
```

- (Optional) Plant recommendations

```
curl -X POST "http://localhost:8000/api/v1/analysis/plant-recommendations/" \
  -H "Content-Type: application/json" \
  -d '{"dataset_id": <DATASET_ID>, "top_n": 10, "criteria":
["bee_attraction","seasonal_availability"]}'
```

- (Optional) Seasonal/site analyses

```
curl -X POST "http://localhost:8000/api/v1/analysis/seasonal/?dataset_id=
<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"include_trends": true, "seasonal_periods": 12}'

curl -X POST "http://localhost:8000/api/v1/analysis/site-comparison/?dataset_id=
<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"metrics": ["total_bees","diversity_index"], "group_by": "site"}'
```

6) Track jobs and fetch results

```
# List jobs
curl "http://localhost:8000/api/v1/analysis/jobs/?dataset_id=<DATASET_ID>"
# Get job status
curl "http://localhost:8000/api/v1/analysis/jobs/<JOB_ID>/status"
# Get job info
curl "http://localhost:8000/api/v1/analysis/jobs/<JOB_ID>"
# Get results
curl "http://localhost:8000/api/v1/analysis/jobs/<JOB_ID>/results"
# (Optional) cancel
curl -X DELETE "http://localhost:8000/api/v1/analysis/jobs/<JOB_ID>"
```

7) Create visualizations

```
# Bee distribution
curl -X POST "http://localhost:8000/api/v1/visualizations/bee-distribution/?
```

```
dataset_id=<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"top_n": 20, "include_percentages": true}'
# Seasonal patterns
curl -X POST "http://localhost:8000/api/v1/visualizations/seasonal-patterns/?
dataset_id=<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"include_trends": true, "seasonal_periods": 12}'
# Site comparison
curl -X POST "http://localhost:8000/api/v1/visualizations/site-comparison/?
dataset_id=<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"metrics": ["total_bees","diversity"], "group_by": "site"}'
# Dashboard
curl -X POST "http://localhost:8000/api/v1/visualizations/dashboard/?dataset_id=
<DATASET_ID>" \
  -H "Content-Type: application/json" \
  -d '{"include_timeline": true, "include_metrics": true}'
```

8) Track visualization tasks and download artifacts

```
# Check task status
curl "http://localhost:8000/api/v1/visualizations/<VIS_ID>/status?task_id=
<TASK_ID>"
# Download info
curl "http://localhost:8000/api/v1/visualizations/<VIS_ID>/download?task_id=
<TASK_ID>"
# Cancel
curl -X DELETE "http://localhost:8000/api/v1/visualizations/<VIS_ID>?task_id=
<TASK_ID>"
# List available viz types
curl "http://localhost:8000/api/v1/visualizations/available-types"
```

9) Monitoring and shutdown insights

```
curl "http://localhost:8000/api/v1/metrics"
curl "http://localhost:8000/api/v1/security/status"
curl "http://localhost:8000/api/v1/shutdown/status"
curl "http://localhost:8000/api/v1/shutdown/metrics"
```

Notes

- Prefer Makefile targets to start services (local, docker, remote) and to check health.
- If auth is enforced, add: `-H "Authorization: Bearer <JWT>"` to protected requests.
- Use returned IDs (`dataset_id`, `job_id`, `visualization_id`, `task_id`) from earlier calls as you proceed.